Tree Based Search Report

Prepared for: COS30019 – Introduction to Artificial Intelligence (AI)
Institution: Swinburne University of Technology

Member Name	Student ID	Contribution	Signature
Alexander Forster	103988837	GBFS, benchmark testing, file parsing, node class, completed research and instruction section documentation.	
Damian Moisidis	104887896	BFS, CUS1, report introduction and proof reading and editing of written work.	DMoisidis
Ngoc Van Nhi Do	105551765	DFS, CUS2, correctness testing, heuristic testing, documentation on features/bugs/missing functionality.	
Renad Ahmed R Aldossary	104799438	A* implementation, report insights, conclusion, and acknowledgements. Proof reading of written work.	

Table of Contents

1. Instructions 2
2. Introduction 3
2.1. Route Finding Problem 3
2.2. Agent Algorithms Developed 4
3. Features, Bugs, and Missing Functionality 4
3.1. Features 5
3.1.1. Graph representation: 5
3.1.2. Search algorithms: 5
3.1.3. Heuristic functions 5
3.1.4. Input and Output 5
3.2. Bugs and Missing Functionality 5
4. Testing 6
4.1. Test cases 6
4.2. Tests 6
4.3. Test results 7
5. Insights 8
5.1. Path Quality 8
5.2. Nodes Visited 8
5.3. Execution Time 8
6. Research 9
7. Conclusion 12
8. Acknowledgements 13
9. References 14

Table of Figures & Tables

Figure 1. Graph of A Start path cost for each scenario *11*
Figure 2. Graph of recursive best first search path cost for each scenario *11*
Figure 3. Graph of greedy best first search path cost for each scenario *11*
Figure 4. Graph of path length of greedy best first search for each scenario *11*
Figure 5. Runtime of each algorithm in each scenario (ms) *12*
Figure 6. CPU usage time of each algorithm in each scenario (ms) *12*
Figure 7. peak memory consumption of each algorithm during each scenario (mb) *13*
Figure 8. Number of nodes expanded by each algorithm during each scenario *13*

Table 1. Overview of test cases 7
Table 2. Overview of correctness tests 8
Table 3. Summary of test results 8
Table 4. Nodes visited across selected test cases 9
Table 5. Execution time across selected test cases 10
Table 6. Extended test cases 10
Table 7. Summary table of average performance metric 12

1. Instructions

There are several files available to use in the solution that serve different purposes how to use them is outlined below.

For main program is a python script called main.py ensure that you are in the Assignment 2A directory and issue the command <**python main.py filename method>** excluding <>. Where filename is the test scenario in the same folder as main.py or it is the path to the test scenario. Method is the algorithm you wish to complete the supplied scenario valid values are BFS, DFS, AS, GBFS, CUS1 or CUS2. The output of this file will be in the format:

filename method
goal number_of_nodes
path

To use the test generator programs in testing, navigate to the testing folder and use the command <python test_generator_manual.py> for the manual GUI graph maker (requires pygame) or <python test_generator_random.py number_of_nodes number_of_edges> for the automatic generator (both require networkx). For the manual generator right click to plot a point, left click to select the point (red indicates it is selected), N to toggle between node numbers and coordinates, press u to toggle between uni directional and bidirectional and press e to export the graph. To draw an edge select two points, select a point and press O to set the origin, to set a destination select a point and press D. For the automatic generator after the python script two numbers must be passed, the first number is the number of nodes in the graph and the second being the number of edges to generate. For both generators generated files will be stored in test_cases\inputs\.

To use extended testing navigate to the testing_extended folder use <python test_generator_extended.py> (networkx is required). The required flags to run are the same as the automatic generator before, however additional flags are available. --width and –height set the max x and y coordinate separately, must be an integer. -- seed sets the seed for the map, type int. --bidirectional_prob is a float that is the probability that an edge will be bidirectional. --cost_noise determines the max random cost to be added to the Manhattan distance, type int. --ensure_connected ensure the graph is weekly connected. --goal_mode has the options of random, far or near, determines how close the destinations are to the goal node. --layout_mode determines the distribution of nodes, can be uniform or clustered. --disconnect_destinations disconnects destinations from the rest of the graph.

2. Introduction

2.1. Route Finding Problem

This report outlines the development and application of six search algorithms to address the route-finding problem. In this problem, an Artificial Intelligence (AI) agent is tasked with finding the optimal path from an origin node to one or more destination nodes on a graph or tree data structure. A tree is a hierarchical structure containing a single root node, where each subsequent node has exactly one parent and edges are unidirectional, making it acyclic. On the other hand, a graph can be cyclic with nodes connected by directed or bidirectional edges that represent various traversable relationships (Cormen et al., 2009). Together, these two data structures form the foundation of the problem the agent must navigate. The problem itself is defined through a text file passed to each algorithm, structured in the following format:

- Nodes, each identified by a number and (x, y) coordinates
- Edges, which define the pathways between nodes and their associated traversal costs.
- Origin, the designated starting node from which the search begins.
- Destination(s), one or more target nodes that the agent must ultimately reach.

2.2. Agent Algorithms Developed

Six algorithms were developed to navigate this challenge, comprising three uninformed and three informed search strategies. The first of the uninformed strategies was Depth-First Search (DFS), which Cormen et al. (2009) describe as an algorithm that seeks to search as deeply as possible into a graph before backtracking upon reaching a dead end. The search continues until the destination node is located, and as a result, can span several trees due to potentially having to restart from multiple sources (Doran & Michie, 1966).

The second uninformed algorithm implemented was Breadth-First Search (BFS), one of the simplest graph search algorithms and the foundation upon which many other well-known algorithms are built. Rather than going deep, it works by expanding the frontier uniformly level by level, producing a breadth-first tree to locate the destination node and determine the shortest path (Cormen et al., 2009).

The third uninformed algorithm is Bi-Directional Breadth-First Search, which was developed as the first custom search algorithm (CUS1), takes the principles of BFS to apply a second search on a reversed map of the node list (graph or tree) to reduce the number of nodes searched. The algorithm detects when a branch from either direction meets a node already visited by the other, at which point the two branches are joined to form the complete path (Kaindl & Kainz, 1997).

The first of the informed search algorithms, Greedy-Best First Search (GBFS) uses an evaluation function expressed as f(n) = h(n), where h(n) is a heuristic that estimates the cost from node n to the destination. GBFS applies a straight-line distance heuristic $h_{SLD}(n)$ to draw a line to the goal, then expand the node that appears closest along that path (Russell & Norvig, 2020).

The second informed search algorithm known as A Star (A*), is a heuristic form of BFS. It is expressed by the function $f(n) = g(n) + h(n)$. It evaluates nodes by combining g(n), the cost to reach the current node, and h(n), the estimated cost from that node to the destination (Portland State University, n.d.).

The third informed algorithm implemented is a Recursive Best-First Search using Manhattan distance heuristics, which for the sake of this report will be referred to as the second custom search algorithm (CUS2). The Manhattan distance formula $|x_1 - x_2| + |y_1 - y_2|$ calculates the distance between two points by summing the absolute differences of their coordinates on a grid-like layout, hence the name (Moudgill, 2025). This heuristic is then applied within a Recursive Best-First Search to determine the destination, number of nodes, and path cost (Korf, 1993). Which in process prunes exhausted paths and prioritising the most promising ones.

3. Features, Bugs, and Missing Functionality

Listed below are the successfully implemented features, along with a description of any unresolved issues or known bugs.

3.1. Features

3.1.1. Graph representation:

Vertices in the graph are represented using a custom Node class, with edges implemented as adjacency lists that support both directionality and edge weights. The complete graph is stored as a dictionary of Node objects.

3.1.2. Search algorithms:

The following six search algorithms were successfully implemented:

- Depth-First Search (DFS)
- Breadth-First Search (BFS)
- Greedy Best-First Search (GBFS)
- A* Search (AS)
- Bi-directional Breadth-First Search for custom search strategy 1 (CUS1)
- Recursive Best-First Search for custom search strategy 2 (CUS2)

All search algorithms accept the graph input, the source node index, and the target node indices as arguments. Each returns a tuple containing the target node reached (if one exists), the number of nodes visited, and the path from the source to the target. Both A* and Recursive Best-First Search are guaranteed to return the shortest path, provided the input graph's edge weights ensure the Manhattan distance heuristic remains admissible (Russell & Norvig, 2020).

3.1.3. Heuristic functions

The following heuristic functions were implemented to support the informed search algorithms:

- Manhattan distance
- Euclidean distance
- Chebyshev distance

3.1.4. Input and Output

The program runs from the console and accepts two command-line arguments, the input file path and the search method name. It then outputs the file path, the search method used, the destination node reached, the number of nodes created, and the path found, all formatted according to the required specification.

3.2. Bugs and Missing Functionality

The implementations for DFS and CUS2 take a recursive approach which could cause an error when it reaches the recursion limit due to the number of recursive calls. While this is unlikely to be an issue for smaller graphs, larger graphs require a greater recursion limit. However, this acts

more as a temporary safeguard than a solution; it is better to implement the algorithms iteratively (or avoid recursive best-first search entirely) if they are known to be handling large graphs (Korf, 1993).

4. Testing

4.1. Test cases

16 input files were manually and automatically generated to test the program. They cover edge cases, different graph types, and different graph sizes. Invalid cases assumed to be out of the scope of this assignment such as negative edge weights, invalid origin and destination nodes, and empty graphs, were not included in testing. The table below provides a brief overview of each test case.

Table 1. Overview of test cases

Test case file	Description
g01.txt	Single-node graph
g02.txt	No solution
g03.txt	Graph with one directed edge
g04.txt	Graph with one undirected edge
g05.txt	Same origin and destination node
g06.txt	Multiple destination nodes with the same path cost
g07.txt	One destination node with multiple paths of the same cost
g08.txt	Cycles in graph
g09.txt	Multiple destination nodes
g10.txt	Shuffled node numbers
g11.txt	Sparse graph
g12.txt	Dense graph
g13.txt	Tiny graph (V=100; E=1,000)
g14.txt	Small graph (V=1,000; E=10,000)
g15.txt	Medium graph (V=10,000; E=100,000)
g16.txt	Large graph (V=100,000; E=1,000,000)

The expected outputs were manually worked out for smaller test cases, particularly test cases 01 to 07. For the rest of the test cases which are too impractical to generate manual solutions, a reference program was utilised to validate our program's outputs by comparison. The reference search algorithms were obtained from NetworkX (Hagberg et al., 2025), a package for network analysis in Python. The reference algorithm for GBFS was adapted from the reference A* algorithm by setting all the edge weights of the graph to 0. However, as a verified and tested reference program could not be found for the bi-directional breadth first search algorithm (CUS1), testing with larger graphs was skipped for this case.

4.2. Tests

Three different tests were written to test each input file with each implemented search method. The tests are described in Table 2 below.

Table 2. Overview of correctness tests

Test number	Test name	Description	Applicable test cases	Applicable methods
1	test_complete	Tests the full required output of each search method, including the goal node reached, the number of nodes created, and the returned path.	01-07	All search methods. Test cases 06 and 07 are skipped for CUS1.
2	test_valid_path	Tests that the returned path is a valid path, i.e. the path is traversable in the given graph.	01-16	All search methods.
3	test_path_match	Tests that the actual path matches the expected path generated by the reference program NetworkX. For the shortest path algorithms AS and CUS2, the test accepts an output which does not match the reference path as long as the path costs are the same.	01-16	All search methods except CUS1.

4.3. Test results

The test results confirm that all implemented search algorithms pass the correctness checks, including edge cases. Each algorithm returns a valid path, or an empty array if all possible paths have been exhausted without reaching a destination node. The DFS, BFS, and GBFS algorithms produce the same paths as the reference implementation, while shortest-path algorithms, AS and CUS2, return either identical paths or alternative paths with the same cost. A summary of test results is provided in Table 3.

Table 3. Summary of test results

Test	BFS	DFS	GBFS	A*	CUS1	CUS2
1	Pass	Pass	Pass	Pass	Pass test cases 01-05; 06-07 skipped	Pass
2	Pass	Pass	Pass	Pass	Pass	Pass
3	Pass	Pass	Pass	Pass	Skip	Pass

The results of each test run, including both the actual and expected outputs, are recorded as CSV files in the 'testing/' directory of the submitted codebase: test_complete.csv, test_valid_path.csv, and test_path_match.csv.

5. Insights

The six search algorithms implemented in this project each exhibit distinct behaviours in terms of path quality, nodes visited, and execution time. The data collected from running all algorithms across the 16 test cases reveals clear trade-offs between uninformed and informed strategies, as well as between path optimality and computational efficiency.

5.1. Path Quality

DFS does not guarantee optimal paths. This is most visible at scale. In g14 (V=1,000), DFS returned a path of length 367 while BFS, CUS1, and CUS2 all returned paths of length 4. BFS guarantees the fewest edges but not lowest cost (Cormen et al., 2009). A* and CUS2 are the only algorithms that guarantee the lowest-cost path, confirmed across all 16 test cases via comparison against the NetworkX reference implementation (Hagberg et al., 2025).

5.2. Nodes Visited

Bidirectional BFS (CUS1) was consistently the most node-efficient algorithm. In g13 (V=100) it visited only 2 nodes versus BFS's 15 and DFS's 41. In g16 (V=100,000) it visited just 65 nodes compared to BFS's 26,068. This efficiency comes from searching simultaneously from both origin and destination and terminating at intersection (Kaindl & Kainz, 1997). CUS2 visited fewer nodes than DFS and BFS on larger graphs due to its pruning behaviour, but more than CUS1 due to backtracking (Korf, 1993).

Table 4. Nodes visited across selected test cases

Test case	DFS	BFS	CUS1	CUS2
g13(V=100)	41	15	2	9
g14(V=1,000)	367	196	4	29
g15(V=10,000)	904	524	9	243
g16(V=100,000)	24,954	26,068	65	5,369

5.3. Execution Time

DFS and BFS were the fastest algorithms due to their simple data structures (Cormen et al., 2009). Despite visiting far fewer nodes, CUS1 was slower on large graphs (582ms on g16 vs 104.7ms for DFS) due to the overhead of maintaining two simultaneous frontiers. CUS2 was the slowest, taking approximately 29 seconds on g16, reflecting the repeated backtracking inherent in recursive best-first search. As Russell and Norvig (2020) note, CUS2, which as mentioned previously is recursive best-first search, may regenerate nodes many times before converging, making it impractical for very large graphs.

Table 5. Execution time across selected test cases

Test case	DFS	BFS	CUS1	CUS2
g14(V=1,000)	0.5ms	0.5ms	1.4ms	1.7ms
g15(V=10,000)	2.4ms	2.2ms	22.5ms	160.1ms
g16(V=100,000)	104.7ms	158.3ms	582.0ms	29,266.6ms

6. Research

To extend this project further, it was decided to conduct additional testing of the algorithms as well as to investigate the impact of heuristic functions on the informed algorithms. In order to achieve this additional test cases were created using an altered version of the test generator, this version allowed for finer control over the variables to generate the graph, the new cases can be seen in the table below, cases 11 to 16 were reused as they were deemed sufficiently complex.

Table 6. Extended test cases

Test case file	Description
g17.txt	Sparse graph, 1000 nodes 3000 edges
g18.txt	Connected dense graph, 1000 nodes 20000 edges
g19.txt	Far destination graph, 1000 nodes 5000 edges
g20.txt	Near-destination graph, 1000 nodes 5000 edges
g21.txt	Larger unsolvable graph, 1000 nodes 5000 edges
g22.txt	Graph with clustered nodes, 1000 nodes 5000 edges
g23.txt	Highly bidirectional graph, 1000 nodes 5000 edges
g24.txt	Mostly directed graph, 1000 nodes 5000 edges
g25.txt	Low-cost noise graph, 1000 nodes 5000 edges
g26.txt	High-cost noise graph, 1000 nodes 5000 edges
g27.txt	Large test case dense, 1000 nodes 30000 edges
g28.txt	Large dense disconnected graph, 1000 nodes 30000 edges

For the heuristic testing each of the informed algorithms (GBFS, A-star and CUS2) were tested on the solvable test cases using each heuristic option, these options were Manhattan distance, Euclidean distance and Chebyshev distance. From these tests it was found that for a star and recursive best first search was not affected at all by which heuristic function was used. Whereas greedy best first search was impacted depending on the function used. With Chebyshev performing well in the high-cost scenario as well as when the destination was close to the origin but had poor performance in the highly random nature of scenario g16. Euclidean distance had a mostly middling performance, unable to surpass Manhattan or Chebyshev distances. It did however perform the worst in the low-cost scenario of g25. Manhattan distance did perform the best overall in finding the path with the lowest cost however, it did perform the worst in the high-costed scenario. It was also found that this pattern was also consistent with the path length, measured by the number of nodes. The resulting graphs of each search method can be seen below.

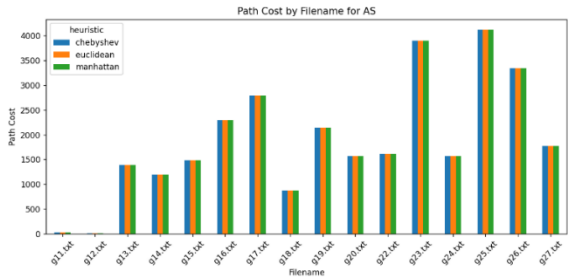

Figure 1. Graph of A Start path cost for each scenario

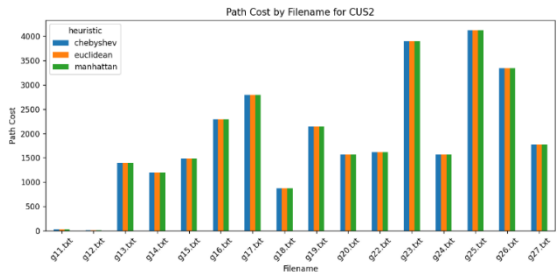

Figure 2. Graph of recursive best first search path cost for each scenario

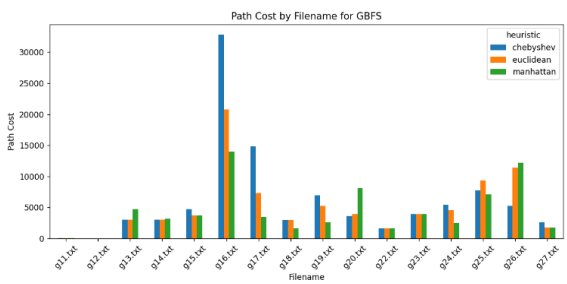

Figure 3. Graph of greedy best first search path cost for each scenario

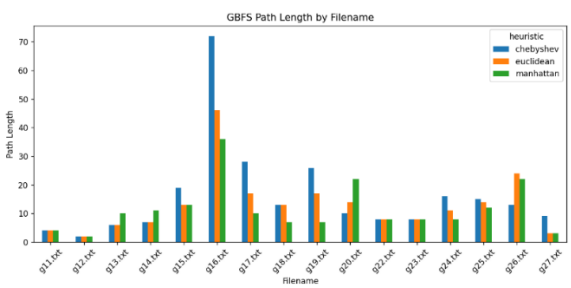

Figure 4. Graph of path length of greedy best first search for each scenario

For further testing it was decided to also measure additional performance indicators, such as runtime, CPU utilisation time, peak memory usage and number of nodes visited. To properly measure these variables each test was conducted 3 times, the average was then calculated, the output of which can be seen in the table and graphs below. From this testing it was found that the recursive best first search (CUS2), while able to find the best path in terms of cost it was the most resource intensive of the algorithms. Having the highest runtime, CPU time and memory usage. The runtime in the large unsolvable cases (g21 and g28) was found to be greater than 20 minutes and due to this it was determined to exclude it from those scenarios. This runtime is attributed to the fact that for large networks unsolvable

networks the algorithm may visited the same node multiple times so long as it not on the same search path. So, while not infinite, can take a very long time to complete. Further it was also found that CUS1 performed the best in the number of nodes that it had expanded to, but at the cost of not finding the cheapest path and had the second longest run time. Greedy best first search found the solution the fastest in terms of runtime but was the second largest in average path cost, only beaten by depth first search.

Table 7. Summary table of average performance metric

method	runtime_ms	cpu_time_ms	peak_memory_mb	nodes_expanded	path_cost
AS	30.374	21.123	561.169	812.944	1882.188
BFS	17.291	6.655	561.155	1941.111	2741.75
CUS1	32.767	23.148	561.953	13.222	2666.375
CUS2	3387.044	3370.768	567.186	467.938	1882.188
DFS	13.65	4.34	561.173	1849.667	1280511
GBFS	16.761	7.523	560.978	442.944	4402.25

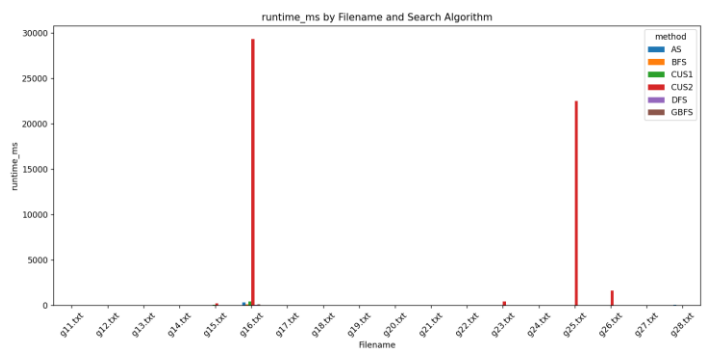

Figure 5. Runtime of each algorithm in each scenario (ms)

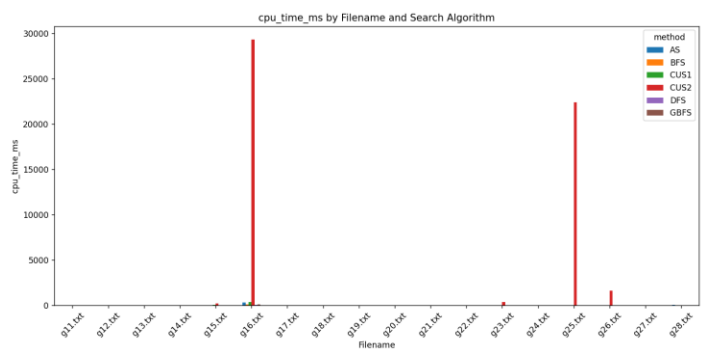

Figure 6. CPU usage time of each algorithm in each scenario (ms)

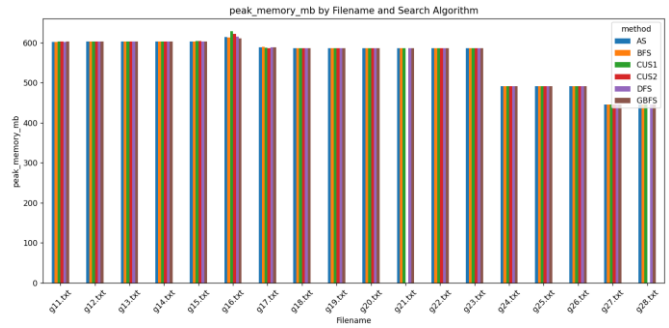

Figure 7. peak memory consumption of each algorithm during each scenario (mb)

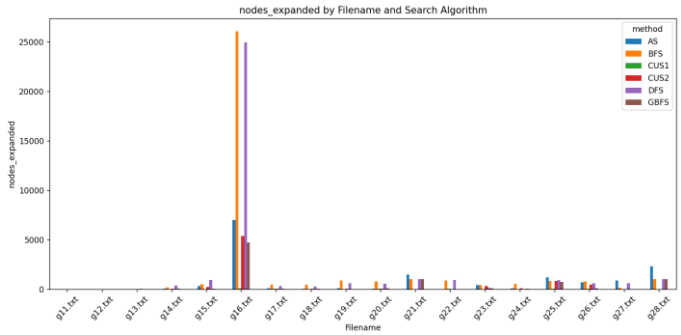

Figure 8. Number of nodes expanded by each algorithm during each scenario

7. Conclusion

This project demonstrated that the choice of search algorithm has significant implications for both the quality of the solution found and the computational resources required. Through the implementation and testing of six algorithms across 16 test cases ranging from single-node graphs to large graphs with 100,000 nodes and 1,000,000 edges, clear patterns emerged regarding the strengths and weaknesses of each approach.

The results show that no single algorithm is the definitive best choice for the route-finding problem. Instead, the most appropriate algorithm depends on the priorities of the application. If path optimality is the primary concern, informed algorithms such as A* and CUS2 are necessary as they are the only ones that guarantee the lowest-cost path. In cases of speed and simplicity being prioritised, DFS and BFS perform well with minimal overhead. And if node efficiency is the priority, CUS1 proved remarkably effective, visiting as few as 65 nodes on a graph of 100,000 nodes by exploiting its bidirectional search structure (Kaindl & Kainz, 1997).

A notable finding is that visiting fewer nodes does not always correspond to faster execution. CUS1 visited far fewer nodes than DFS and BFS on large graphs yet was slower in execution time due to the overhead of maintaining two simultaneous search frontiers. Similarly, CUS2 visited fewer nodes than DFS on large graphs but was significantly slower due to repeated

backtracking (Korf, 1993). These results highlight that node count and execution time are related but distinct measures of efficiency.

Performance could be improved in several ways. Converting the recursive implementations of DFS and CUS2 to iterative alternatives would make them more reliable on large graphs by removing the risk of hitting Python's recursion limit (Korf, 1993). Beyond that, extending CUS1 with cost awareness could allow it to find optimal paths while retaining its node efficiency (Kaindl & Kainz, 1997). Finally, experimenting with different heuristic functions such as Euclidean or Chebyshev distance may also yield performance gains depending on the geometry of the graph being searched (Russell & Norvig, 2020).

8. Acknowledgements

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. Used as the primary reference for understanding and implementing DFS and BFS, including pseudocode for graph traversal, frontier expansion, and path reconstruction.

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson. Used as the main reference for informed search strategies, providing formal definitions for GBFS, A*, and the theoretical limitations of Recursive Best-First Search.

Kaindl, H., & Kainz, G. (1997). Bidirectional heuristic search reconsidered. Journal of Artificial Intelligence Research, 7, 283–317. Provided insight into how bidirectional frontiers are managed and how the meeting condition is used to reconstruct the full path.

Korf, R. E. (1993). Linear-space best-first search. Artificial Intelligence, 62(1), 41–78. Primary reference for implementing CUS2, describing the RBFS mechanism including f-value limits, backtracking conditions, and linear space constraints.

Hart, P., Nilsson, N., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107. Referenced for the theoretical guarantees of A*, supporting implementation decisions around the f(n) = g(n) + h(n) evaluation function and admissibility requirements.

Moudgill, K. (2025). Navigating the grid: Understanding the Manhattan distance. Medium. Used to support understanding of the Manhattan distance heuristic applied in CUS2, clarifying its suitability for coordinate-based graph layouts.

Portland State University. (n.d.). A.I.: Informed search algorithms — Chapter III: Part deux [Lecture slides]. Consulted as a supplementary reference for understanding how GBFS and A* evaluate and expand nodes.

Doran, J., & Michie, D. (1966). Experiments with the graph traverser program. Proceedings of the Royal Society of London, 294, 235–259. Referenced for historical context on DFS behaviour, supporting the discussion of its tendency to explore deeply and span multiple trees.

NetworkX — Hagberg, A., Schult, D., & Swart, P. NetworkX (Version 3.4.2) [Computer software]. https://networkx.org Used as a reference implementation to validate the correctness of all search algorithms across the 16 test cases by cross-checking paths and costs.

9. References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Doran, J., & Michie, D. (1966). Experiments with the graph traverser program. *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences, 294*, 235–259. https://api.semanticscholar.org/CorpusID:21698093

Hagberg, A., Schult, D., & Swart, P. (2008). *NetworkX* (Version 3.4.2) [Computer software]. https://networkx.org

Hart, P., Nilsson, N., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/tssc.1968.300136

Kaindl, H., & Kainz, G. (1997). Bidirectional heuristic search reconsidered. *Journal of Artificial Intelligence Research, 7*, 283–317. https://doi.org/10.1613/jair.460

Korf, R. E. (1993). Linear-space best-first search. *Artificial Intelligence, 62*(1), 41–78. https://doi.org/10.1016/0004-3702(93)90045-d

Moudgill, K. (2025). Navigating the grid: Understanding the Manhattan distance. *Medium*. https://medium.com/@moudgillkarnvir/navigating-the-grid-understanding-the-manhattan-distance-382d264cd13e

Portland State University. (n.d.). *A.I.: Informed search algorithms — Chapter III: Part deux* [Lecture slides]. https://web.pdx.edu/~arhodes/ai8.pdf

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson. https://aima.cs.berkeley.edu/